

---

# A METHOD FOR CALCULATING THE UNRESTRICTED DAMERAU-LEVENSHTEIN EDIT DISTANCE USING BIT-LEVEL PARALLELISM

Technical report 2022-02-23

Heikki Vesalainen, Corsearch Inc, [heikki.vesalainen@corsearch.com](mailto:heikki.vesalainen@corsearch.com)

## BACKGROUND

### DAMERAU-LEVENSHTEIN EDIT DISTANCE

The Damerau-Levenshtein edit distance (Damerau-distance) is a measure for the difference between two letter strings measured in the minimum number of edits that must be performed to transform the first string into the second. The allowed edits are:

- Inserting one letter
- Deleting one letter
- Changing one letter to another
- Transposing two adjacent letters

In the restricted version of the Damerau-Levenshtein (also called the *optimal string alignment* distance), the transposition is only allowed for a pair of letters that is adjacent in both strings before any insertions or deletions. In the unrestricted version of the algorithm, the transposition can be preceded by deletions or followed by insertions. The original Levenshtein algorithm, on the other hand, didn't allow transpositions at all.

For example, the difference between the strings CORSEARCH and KRSAERCH could be expressed as the following edits:

1. CRSEARCH: deleted the O
2. KRSEARCH: changed the initial C to K
3. KRSAERCH: transposed EA

This would be counted as three edits and hence we would say the distance between the two strings is 3. In this example, the EA → AE transposition could be done without any deletions or insertions between the letters.

However, consider the example FOUND vs. FNOD, here the transposition of ON → NO cannot be done before the intervening U has been deleted. This means, the restricted and the unrestricted versions yield a different score. Below is the comparison of how the restricted and the unrestricted version would see the edits.

| Restricted version        | Unrestricted version      |
|---------------------------|---------------------------|
| FUND: deleted O           | FOND: deleted U           |
| FND: deleted U            | FNOD: transposed ON       |
| FNOD: inserted O          |                           |
| <i>Number of edits: 3</i> | <i>Number of edits: 2</i> |

The original Levenshtein algorithm and the unrestricted Damerau-Levenshtein are metric, i.e., they meet the following three conditions:

1.  $d(A, B) = 0 \Leftrightarrow A = B$
2.  $d(A, B) = d(B, A)$
3.  $d(A, C) \leq d(A, B) + d(B, C)$

However, the restricted Damerau-Levenshtein does not fulfil the third condition, the triangle inequality, and hence it is not a metric.

A naïve implementation of the Damerau-Levenshtein has a running-time of  $O(M \cdot N \cdot \min(M, N))$ , but using a more sophisticated algorithm following the idea by (Wagner & Lowrance, 1975) the algorithm can be rewritten to run in  $O(M \cdot N)$  time.

The algorithm presented in this paper implements the Damerau-Levenshtein distance.

## BIT-LEVEL PARALLELISM

A general implementation of the Damerau-Levenshtein algorithm is usually done using a memory array of size  $M \cdot N$  for storing the intermediate results and computing those results in two nested loops, one looping through the characters of the first string (of length  $M$ ), and another one looping through the second string (of length  $N$ ). At each step of the algorithm, a decision is made about the score for one of the  $M \cdot N$  letter pairs based on whether the compared letters are the same or not.

An algorithm using the methods of bit-level parallelism, or bit parallelism for short, would instead use the bits of a CPU register to represent a state machine on which several states are operated at the same time using primarily bit-wise operations. Hence a bit-parallel algorithm can cut the number of steps required to execute an algorithm by the width  $W$  of the available processor word, allowing for an algorithm that has a running time of  $O(M/W \cdot N)$ .

The algorithm presented in this paper exploits the idea of bit-level parallelism to provide a method by which comparing a pattern of length  $M \leq 64$  against an input of any length  $N$  can be done in  $O(N)$  time. This is a clear speed-up compared to the state of the art, which is  $O(M \cdot N)$ .

## PRIOR WORK

The present algorithm builds on the previous ideas presented by Gene Myers and Heikki Hyyrö. Myers introduces in (Myers, 1999) the idea of using bit-parallelism for computing the edit distance between two strings. In his paper he shows how to implement the Levenshtein-algorithm as a bit-parallel version.

---

```
1   Precompute peq[σ]
      as peg[s](i) ≡ (pi = s)
2   vp = -1 // all bits set
3   vm = 0
4   score = m

5   for j = 1, 2, 3, ..., n do
6       eq = peq[tj]
7       vx = eq | vm
8       hx = ((eq & vp) + vp) ^ vp | eq
9       hp = vm | ~(hx | vp)
10      hm = vp & hx

11      if hp & (1 << (m-1)) then
12          score += 1
13      else if hm & (1 << (m-1)) then
14          score -= 1

15      hp = (hp << 1) | 1
16      vp = (hm << 1) | ~(vx | hp)
17      vm = hp & vx
```

---

*Listing 1: Adaptation of the algorithm of Myers for calculating the Levenshtein distance between the strings  $p$  of length  $m \leq W$  and  $t$  of length  $n$  (not restricted), where  $W$  is the bit width of the available CPU registers.*

In the Myers algorithm, the `peq` is a look-up table from the alphabet of  $p$  and  $t$  to a vector having a bit set at every position where the pattern  $p$  has the corresponding letter. Bit vectors  $hp$  and  $hm$  together represent the *horizontal delta vector's* plus and minus components, and respectively  $vp$  and  $vm$  for the vertical case. The vectors  $vx$  and  $hx$  form the *X-factor*. See (Myers, 1999) for the explanation and proof of the algorithm.

It should be noted that the algorithm proposed by Myers is for searching an approximate substring  $p$  from a longer string  $t$  and stopping at the first occurrence that is within  $k$  edits of the pattern. The above adaptation removes this stopping condition and modifies line 15 to add the insertion cost (`| 1`), making it compute the Levenshtein distance between the strings  $p$  and  $t$ .

Hyyrö analyses the work of Myers in (Hyyrö, 2001), and builds on his work to introduce a bit-parallel version of the restricted Damerau-Levenshtein algorithm for approximate string matching in (Hyyrö, 2003) and (Hyyrö, 2005).

In the final version of 2005, Hyyrö had two modifications with respect to the original algorithm by Myers. The vertical and horizontal *X-factor*  $vx$  and  $hx$  on listing 1 rows 7–8 is replaced by a *diagonal zero delta* bit vector  $d$  and a new *match or transposition* vector  $tc$  is introduced.

The central change of the algorithm is to define the condition on which a transposition is possible in the restricted case on row 9.

---

```

1   Precompute peq[o]
      as peg[s](i)  $\equiv$  ( $p_i = s$ )
2   vp = -1 // all bits set
3   vm = 0
4   eq_1 = 0
5   tc = 0
6   score = m

7   for j = 1, 2, 3, ..., n do
8       eq = peq[tj]
9       tc = eq | (((tc & eq) << 1) & eq_1)
10      d = (((tc & vp) + vp) ^ vp) | tc | vm
11      hp = vm | ~(d | vp)
12      hm = vp & d

13      if hp & (1 << (m-1)) then
14          score += 1
15      else if hm & (1 << (m-1)) then
16          score -= 1

17      hp = (hp << 1) | 1
18      vp = (hm << 1) | ~(vx | hp)
19      vm = hp & d
20      eq_1 = eq

```

---

*Listing 2: Adaptation of the algorithm of Hyvärö for calculating the restricted Levenshtein-Damerau distance.*

## THE UNRESTRICTED BIT-PARALLEL DAMERAU-LEVENSHTEIN ALGORITHM

To be able to implement the unrestricted version of the algorithm, it is important to understand when a transposition would yield the lowest score.

Let  $|D|$  and  $|I|$  represent the number of deletions and insertions, respectively, between the transposed pair.

**Lemma 1:** A transposition needs to be considered only when  $|D| = 0$  and  $|I| = 0$ , or both. In all other cases the transposition has the same or higher cost than covering the area with substitutions, inserts and deletes.

**Proof 1:** let  $aDb$  and  $bIa$  be two strings consisting of the letters  $a, b$  and the strings  $D, I$  not containing either letter. The maximum distance between them is  $\max(|aDb|, |bIa|)$ . This is because the first  $\min(|aDb|, |bIa|)$  letters can be covered with substitutions and the remaining  $\max(|aDb|, |bIa|) - \min(|aDb|, |bIa|)$  with inserts or deletes, depending on which string is longer. When  $|I| = 0$  or  $|D| = 0$ , i.e.,  $|I| + |D| = \max(|I|, |D|)$  the other string has length 2 and the other string has length  $2 + \max(|I|, |D|)$ . Without a transposition operation their maximum distance would thus be  $2 + \max(|I|, |D|)$ .

With transposition, the distance is  $1 + |I| + |D| = 1 + \max(|I|, |D|)$  and the transposition is applicable. However, when  $|I| > 0$  and  $|D| > 0$ , the cost of covering the string with substitutions and inserts/deletes is no longer  $1 + \max(|I|, |D|)$ , but  $1 + |I| + |D| = 1 + \min(|I|, |D|) + \max(|I|, |D|)$ . One can now immediately see that if  $|I| = |D| > 0$ , this becomes  $|I| + 2 \cdot \max(|I|, |D|)$  and if  $|I| \neq |D|, |I| > 0, |D| > 0$ , then this is at least  $1 + 1 + \max(|I|, |D|) = 2 + \max(|I|, |D|)$ . Hence, in the latter case, the transposition is at least as costly as the maximum score using only substitutions, inserts and deletes.

**Lemma 2:** When there are multiple transpositions possible in the same substring, the most local one (with least deletes or inserts) yields the lowest score.

**Proof 2:** The case can be expressed as the distance between  $aDb, ba$ , where  $D$  contained  $a$  or  $b$ .

Let us first consider if  $D$  contained  $b$ , we could write  $D = HbT$ , thus we are concerned with the distance  $aHbTb, ba$ . The length  $|D| = |HbT| = |H| + |T| + 1$ , where both  $H$  and  $T$  are possibly empty.

Using the transposition rule for the first and final letters, the edit distance is  $|D| + 1$ . But the same could be had by applying the transpose rule for  $aHb$  and then inserting  $Tb$ , yielding the score  $|H| + 1 + |T| + 1$ , which is  $|D| + 1$ . Similarly, if  $D$  contained  $a$  and v.v. for insertion instead of deletion.

Hence, we can always consider just the most local transposition of several possible transpositions in the same substring.

**Lemma 3:** A transposition yields the lowest score only if the transposed characters are not the same (i.e., no match).

**Proof 3:** Consider an edit where this wouldn't be the case  $aDa \rightarrow aa$ , where  $D$  is any string not containing  $a$  (see lemma and proof 2), and which can thus be written as the following edit: delete  $D$  to get  $aa$ , cost is  $|D|$ .

However, using the definition of the transposition, the cost of this would be  $|D| + 1$ , which is higher. Hence the case should not be considered a transposition. Similarly for the v.v. case but using insert in place of delete.

**Lemma 4:** two adjacent transpositions sharing a position yield the same score as using only inserts and deletes.

**Proof 4:** the case can be expressed as distance of  $aXbYc$  to  $cab$ , which, using transpositions would yield the following edits (multiple orders are possible):

- delete  $Y$  to get  $aXbc$ , cost is  $|Y|$
- delete  $X$  to get  $abc$ , cost is  $|X|$
- transpose  $bc$  to get  $acb$ , cost is 1
- transpose  $ac$  to get  $cab$ , cost is 1

However, the same cost can be had with inserts and deletes only:

- delete  $Y$  to get  $aXbc$ , cost is  $|Y|$
- delete  $X$  to get  $abc$ , cost is  $|X|$
- delete  $c$  to get  $ab$ , cost is 1
- insert  $c$  to get  $cab$ , cost is 1

Similarly for the v.v. case but reversing the inserts and deletes.

**Corollary:** From lemma 1, we can say that we only need to consider transpositions where the transposed pair is either on adjacent rows (no deletions) or adjacent columns (no insertions).

From lemma 2, 3, and 4, we can say that we should not consider cases where there is a match at any point within the transposition or the start of the substring was part of an earlier transposition.

From all of the lemmas 1–4 together, we have that for any pair of strings  $XaDb$  and  $YbIa$ , where  $X$  and  $Y$  are any strings of any length,  $a \neq b$  are two characters, and  $I$  and  $D$  are two strings, not containing  $a$  or  $b$  for which  $\min(|D|, |I|) = 0$  (i.e. one of them is the empty string), then the edit distance, if the transposition between  $a$  and  $b$  is to be used, is  $d(XaDb, YbIa) = d(Xa, Yb) + \max(|D|, |I|)$ . Thus, for the *diagonal delta* at the last position to be 0, the sum of the horizontal deltas must be  $|D|$  and the sum of the *vertical deltas* must be  $|I|$ . Furthermore, as there are  $|D|$  and  $|I|$  positions between the transposed pair, it means all the columns or rows must have a  $hp$  or  $vp$  bit set respectively.

The presented algorithm modifies the Hyrrö algorithm in listing 2 by removing the  $tc$  bit vector, changing the definition of the *diagonal zero delta* vector  $d$ , and introducing a *transposition after deletes* bit vector  $txd$  and *transposition before inserts* bit vector  $txi$ , which are combined to be a *transposition allowed* bit vector  $tx$ . Below we describe the computation of these vectors and explain how they work.

$$txd = eq\_1 \ \& \ (((eq \ \& \ \sim tx) \ll 1) + (vp \ \& \ \sim (eq\_1 \mid eq)))$$

A transposition is enabled between two adjacent columns between rows  $i$  and  $i - n$ , when there is a match in the previous column at row  $i$  (bit vector  $eq\_1$ ) and in the current column ( $eq$ ) at row  $i - n$ , where  $n \geq 1$ . Furthermore, the position in the previous row at row  $i - n$  must not have taken part in a previous transposition ( $\sim tx$ ).

Thus, the bit-vectors  $eq\_1$  and  $(eq \ \& \ \sim tx\_1)$  will have a bit set for the two ends of the transposition where  $eq\_1$  has it on the row where the transposition's result should be recorded and  $(eq \ \& \ \sim tx\_1)$  has it on some row  $n$  rows above that. To get the transposition to be recorded, the latter needs to be "transported" to the result row.

If we would be only interested in adjacent transpositions without deletions or insertions between the transposed pair (i.e.,  $n = 1$ ), then a simple shift of one ( $\ll 1$ ) would suffice. However, in the case of two adjacent columns,

we must allow for deletions, i.e., that the rows are separated by  $n > 1$  rows. Furthermore, there could be several transpositions in the same column, each having their own  $n$ .

The trick here is to use the binary addition to *carry* the bit from row  $i - n$  to  $i$  when the transposition would yield the lowest score. From lemma 2, we can say that neither of the transposed characters should have a match in between the transposed pair ( $\sim(eq\_1 \mid eq)$ ) and furthermore, the vertical plus vector ( $vp$ ) should have a bit set for each intermediate position. Hence  $vp \ \& \ \sim(eq\_1 \mid eq)$  yields a vector that has the bits set to *carry* the  $(eq \ \& \ \sim tx\_1) \ll 1$  result to the correct row.

On the other hand,  $txi$  works by keeping a *transposition possible* bit up for a row.

$$txi = ((eq \gg 1) \ \& \ \sim tx) \mid (txi \ \& \ hp)$$

The transposition possible bit is kept on the row  $i$ , where a possible future match will trigger the transposition.

A transposition will be enabled in a future column, if in the current column there was a match in the row below, that is on row  $i + 1$  ( $eq \gg 1$ ) and there was no transposition in the current row  $i$  ( $\sim tx$ ), or when a transposition has already been enabled ( $txi$ ), and the current column has a horizontal plus ( $hp$ ) to carry the bit forward.

Then, when handling the next column,  $txi$  is used for testing if transposition is possible by computing:

$$(txi \ \& \ eq) \ll 1$$

That is, if the row  $i + 1$  has the  $txi$  bit set and a match, then a transposition is possible on the row  $i$ .

The vectors  $txd$  and  $txi$  are then combined as the *transposition allowed* bit vector  $tx$  as

$$tx = txd \mid (txi\_1 \ \& \ eq) \ll 1$$

The complete algorithm is presented in listing 3.

The presented algorithm works for any pattern whose length is at maximum the same as the length of the longest available bit vector that supports addition. This would normally be the widest available processor register. For natural language applications this is more than enough since a normal natural language character count hardly exceeds the common 64-bit width of a modern CPU register.

However, to support longer patterns, one should apply the block model (an adaptation of the *Four Russians* approach) described by (Myers, 1999).

---

```

1    Precompute peq[σ]
      as peg[s](i) ≡ (pi = s)
2    vp = -1 // all bits set
3    vm = 0
4    eq_1 = 0
5    tx = 0
6    txi = 0
7    score = m

8    for j = 1, 2, 3, ..., n do
9        eq = peq[tj]
10       txd = eq_1 & (((eq & ~tx) << 1) + (vp & ~(eq_1 | eq)))
11       tx = txd | (txi & eq) << 1
12       d = (((eq & vp) + vp) ^ vp) | eq | vm | tx
13       hp = vm | ~(d | vp)
14       hm = vp & d
15       txi = ((eq >> 1) & ~tx) | (txi & hp)

16       if hp & (1 << (m-1)) then
17           score += 1
18       else if hm & (1 << (m-1)) then
19           score -= 1

20       hp = (hp << 1) | 1
21       vp = (hm << 1) | ~(d | hp)
22       vm = hp & d
23       eq_1 = eq

```

---

*Listing 3: The unrestricted bit-parallel Damerau-Levenshtein algorithm*

## REFERENCES

- Hyyrö, H. (2001). *Explaining and Extending the Bit-parallel Approximate String Matching Algorithm of Myers*.
- Hyyrö, H. (2003). *A Bit-Vector Algorithm for Computing Levenshtein and Damerau Edit Distances*.
- Hyyrö, H. (2005, June). Bit-Parallel Approximate String Matching Algorithms with Transposition. *Journal of Discrete Algorithms*, 3(2-4), 215-229.
- Myers, G. (1999, May). A Fast Bit-Vector Algorithm for Approximate String Matching Based on Dynamic Programming. *Journal of the ACM*, 46(3), 395-415.
- Ukkonen, E. (1992, January). Approximate string-matching with q-grams and maximal matches. *Theoretical Computer Science*, 92(1), 191-211.
- Ukkonen, E. (1993). Approximate string-matching over suffix trees. *Proceedings of the 4th Symposium on Combinatorial Pattern Matching*, 684, pp. 28-242.
- Wagner, R. A., & Lowrance, R. (1975, April). An Extension of the String-to-String Correction Problem. *Journal of the ACM*, 22(2), 177-183.